

Scanner Wiring

Pin-for-pin against `scanner/include/pins.h`. Power first, because that is what actually kills these builds — and the failure looks like a firmware bug for days.

READ BEFORE CONNECTING ANYTHING

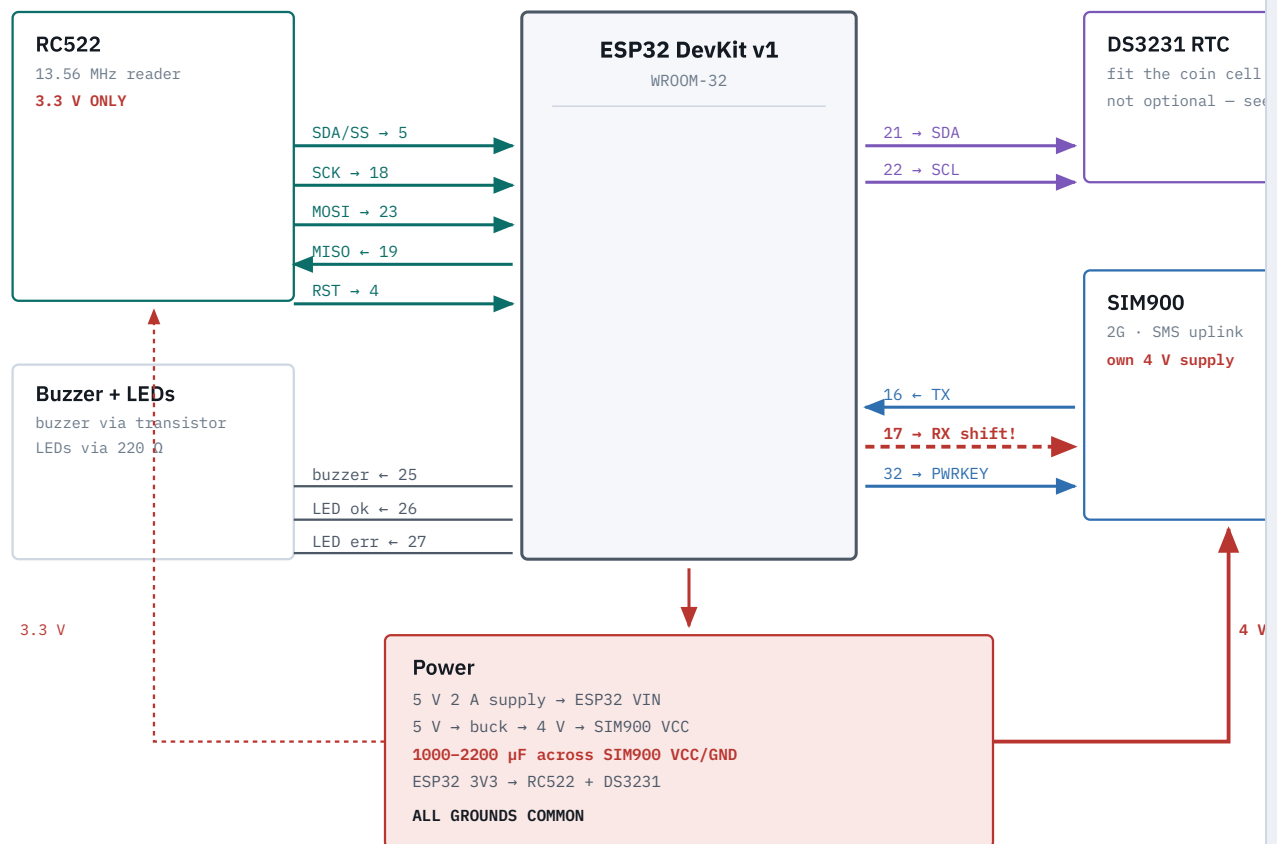
The SIM900 draws up to 2 A in transmit bursts at 3.4–4.4 V. It cannot run from the ESP32's 3V3 pin, from the DevKit's 5V pin, or straight off USB. Give it its own buck regulator to ~4 V, with a **1000–2200 μ F electrolytic across its VCC/GND, physically close to the module**, and short thick wires.

Without that capacitor the transmit burst collapses the rail, the ESP32 browns out and resets, and you get random reboots that look exactly like a firmware fault. Being mains powered removes the battery budget, not the current spike.

The RC522 is 3.3 V only. 5 V destroys it, permanently and immediately.

01 Connection diagram

— SPI — card reader — I²C — clock — UART2 — SIM900 — Power — Needs level shifting



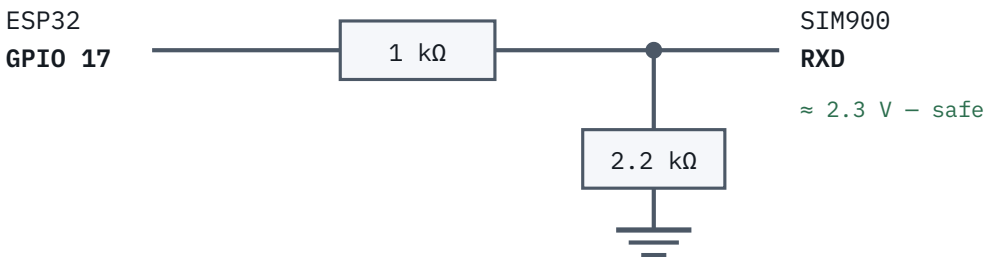
Note the two supplies. The SIM900 does not share a rail with anything else, and its ground still ties back to the ESP32's – a modem on a separate supply with a floating ground will enumerate on the bench and then fail unpredictably in the field.

02 Connection table

MODULE	MODULE PIN	ESP32	BUS	NOTE
RC522	SDA / SS	GPIO 5	SPI	Boot strapping pin; idle-high SS is fine
RC522	SCK	GPIO 18	SPI	VSPI default
RC522	MOSI	GPIO 23	SPI	

MODULE	MODULE PIN	ESP32	BUS	NOTE
RC522	MISO	GPIO 19	SPI	
RC522	RST	GPIO 4	SPI	Moved off 22 so I ² C keeps its standard pair
RC522	3.3V / GND	3V3 / GND	PWR	Never 5 V
DS3231	SDA	GPIO 21	I ² C	4.7 kΩ pull-ups if your board lacks them
DS3231	SCL	GPIO 22	I ² C	
DS3231	VCC / GND	3V3 / GND	PWR	Fit the backup cell, or offline taps get refused
SIM900	TXD	GPIO 16	UART2	~2.8 V out reads as HIGH; no shifting needed
SIM900	RXD	GPIO 17	UART2	Level-shift. Its RX is not 3.3 V tolerant
SIM900	PWRKEY	GPIO 32	UART2	Hold low ~1.2 s to toggle power
SIM900	VCC / GND	4 V rail / GND	PWR	Separate supply + bulk cap
Buzzer	signal	GPIO 25	—	Via NPN/MOSFET, not straight off the pin
LED green (<i>or RGB G leg</i>)	anode	GPIO 26	—	220 Ω series
LED red (<i>or RGB R leg</i>)	anode	GPIO 27	—	220 Ω series
RGB B leg	anode	GPIO 13	—	220 Ω series. Optional — set <code>LED_IS_RGB</code>
RGB common	common leg	GND <i>or</i> 3V3	PWR	Cathode → GND. Anode → 3V3 and set <code>LED_COMMON_ANODE</code>
WiFi	—	—	PWR	Configurable via captive portal at 192.168.4.1 (WiFiManager). AP+STA mode

Only one line needs it. The SIM900's RX input is not 3.3 V tolerant, and driving it directly works on the bench for a while and then degrades the pin. A two-resistor divider is enough for a one-way 9600-baud line.

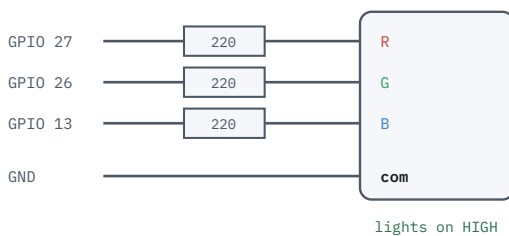


$3.3 \text{ V} \times 2.2 / (1 + 2.2) \approx 2.3 \text{ V}$. The other direction needs nothing: the SIM900's $\sim 2.8 \text{ V}$ output already reads as a logic HIGH on the ESP32.

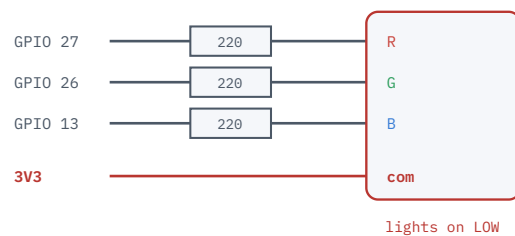
3B The 4-pin RGB LED

Four pins means three colour legs and one common. Which kind you have decides the wiring *and* a firmware constant, and getting it wrong inverts every cue — the LED sits lit at idle and goes dark when something happens.

COMMON CATHODE — the default
`LED_COMMON_ANODE = false`



COMMON ANODE — inverted
`LED_COMMON_ANODE = true`



The longest leg is the common. Leg order is usually R · com · G · B.

Telling them apart: put the multimeter in diode mode, black probe on the long leg. If a colour lights, it is **common cathode**. Nothing? Swap the probes — if it lights with red on the long leg, it is **common anode**. No meter: common leg to GND through 220 Ω, touch another leg to 3V3. Lights → cathode.

SYMPTOM OF PICKING THE WRONG ONE

Every LED lit at idle and dark during cues — the exact inverse of what you expect. The firmware is fine; flip `LED_COMMON_ANODE`. It is one constant because every write goes through a single helper in `ui.cpp`, which is what stops this becoming a hunt through a dozen call sites.

COLOUR	MEANS
Green, steady	Online, queue empty — the gate is healthy
Green, chirp	Enrolled card accepted
Amber (R+G)	Card is not on the roster — still recorded
Blue, blinking	Buffering: taps stored, network down
Red, blinking	No usable clock. Taps are being refused
Red, long tone	Could not store the tap — flash full

04 Bring-up, one module at a time

Wiring all four then flashing the full firmware means any fault presents as "nothing works". Bring them up in this order — each step is independently verifiable.

01 ESP32 alone

Blink GPIO 26 and 27, sound GPIO 25. Confirms the board, the supply and your feedback wiring before anything else can confuse the picture.

02 DS3231 next, not last

Scan I²C and read the time. It is first because the scanner *refuses taps* without a trustworthy clock, so a missing RTC would otherwise look like a broken reader.

03 RC522

Dump a card UID over serial. If you get all-zeros or `FFFFFFFF`, it is wiring or a 5 V mistake — not the card.

04 SIM900 on its own supply — *skippable*

Before any code: power it, watch the status LED, then `AT` → `OK` , `AT+CREG?` → `0,1` or `0,5` , `AT+CSQ` ≥ 10. Send one SMS by hand with `AT+CMGF=1` and `AT+CMGS` .

Not fitted yet? Leave `SIM900_PRESENT = false` and skip to step 5. Taps are still read, stored and delivered over Wi-Fi, and the *server* texts the parent instead — nothing is lost and nothing is duplicated.

05 Full firmware

Only now. Serial should print `scanner up | rtc=1 roster=N queued=0 sms=not fitted (server will send)` , or `sms=ready` once the modem is in.

05 Symptom → cause

SYMPTOM	ALMOST ALWAYS
ESP32 reboots when a text is sent	Missing or undersized bulk cap on the SIM900 rail. The single most common fault on these builds
SIM900 never answers <code>AT</code>	PWRKEY not pulsed, or TX/RX swapped. Try swapping them — it costs nothing
Answers <code>AT</code> , garbled replies	Baud mismatch, or GPIO 17 driven without the divider
<code>AT+CREG?</code> returns <code>0,2</code> forever	Searching: no 2G coverage, SIM not registered under RA 11934, or load expired
Reader gives all-zero UIDs	RC522 on 5 V (likely dead), or MISO/MOSI swapped
Reader works cold, dies after a minute	RC522 sharing the SIM900's sagging rail. Put it on 3V3
Taps refused, red LED blinking	DS3231 missing or its coin cell is flat. By design — a wrong timestamp corrupts the register silently
Everything works on USB, fails on the enclosure supply	Supply cannot deliver the 2 A burst, or grounds are not common

Matches `scanner/include/pins.h` at the time of writing — if you change a pin, change it there first and regenerate this. GPIO 5 is a boot strapping pin (fine for an idle-high SS) and GPIO 12 is deliberately

unused, since pulling it high at boot sets the flash voltage and bricks startup.